

AI-Based Solution for Analysing, Benchmarking and Quality Monitoring
of Cybersecurity Audit Reports

TECHNICAL MANUAL

Objective 1: Standardisation of Cybersecurity Audit Report Format

By,
Niketa, AI Engineer

Contents

1. Introduction
 2. System Architecture
 3. Hardware and Software Requirements
 5. Module Descriptions
 6. Data Design
 7. Dependencies and Third-Party Components
 8. Installation and Configuration
 9. Interfaces
 10. Security Architecture
 11. Performance and Capacity
- Appendix A: Environment Variables
- Appendix B: Master Schema Field Reference
- Appendix C: Glossary

1. Introduction

1.1 Purpose

This manual describes the design and construction of the Report-to-JSON pipeline delivered under Objective 1 of the project. It is intended to let a new engineer install, configure, maintain, test and extend the system without help from the original developers. Operating instructions for end users are in the separate User Manual.

1.2 Scope

The manual covers the software in the repository: the three-stage conversion pipeline, the desktop, web and command-line front ends, the security pre-gate modules, the Master Schema and the model-loading tools. It does not cover the later project objectives (dashboard, benchmarking, agency monitoring), which will consume the JSON produced here.

2. System Architecture

2.1 Architectural Overview

The system is a Python application that runs entirely on one host. It has no database and no network service other than the optional web interface. A report goes in as a file; a standardised JSON file, an intermediate Markdown transcript and a folder of extracted images come out. Figure 1 shows the logical layers.

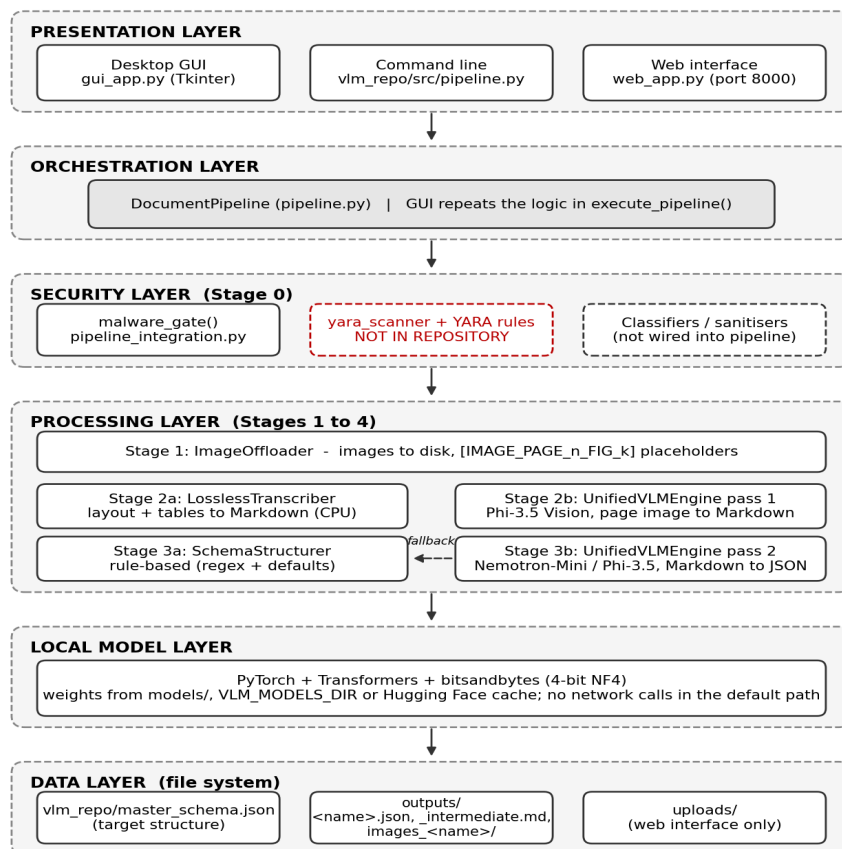


Figure 1: Logical architecture of the Report-to-JSON pipeline

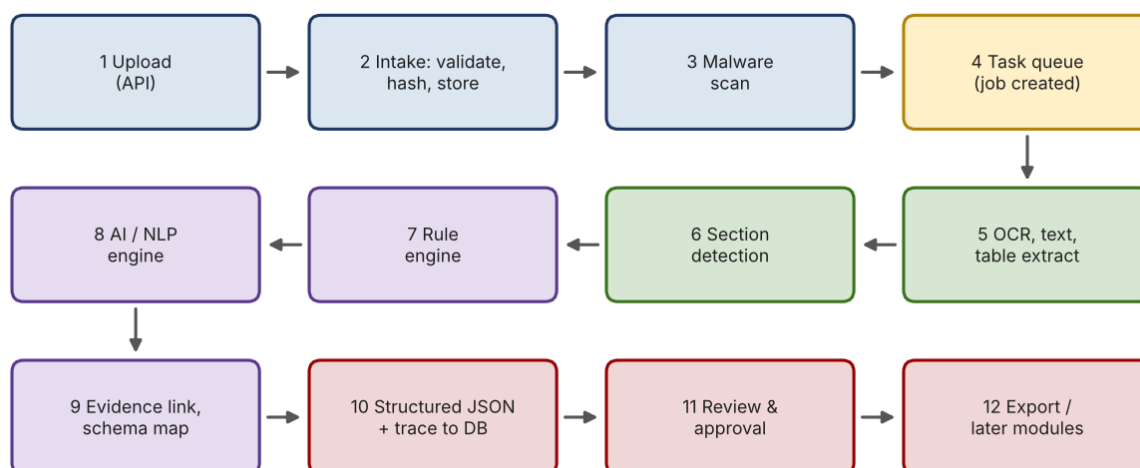
The item with a red dashed outline is referenced by the code but is not present in the repository (see Section 14). Arrows show the order of control, not data volume.

2.2 Layers and Responsibilities

Layer	Responsibility
Presentation	Accept a file and options, show progress and results
Orchestration	Run stages in order, handle errors, write outputs. The GUI carries its own copy of the orchestration logic
Security	Screen and sanitise input before parsing
Processing	Stages 1 to 3 of the conversion
Local model	Locate, load and run the models in-process with 4-bit quantisation
Data	Target structure and file storage

2.3 Processing Flow

Figure 3 follows a single report from upload to approved structured output. Each stage reads its input from the object store or database, writes its result back, and emits a status event to the audit log, which is what the user sees in the Processing Queue



Each worker reads from and writes to the object store and database; status events are written to the audit log at every step.

Figure 2: Processing flow for a single report

2.4 Design Principles Found in the Code

- Decoupled stages. Images, transcription and structuring are separate steps connected by files (images folder, intermediate Markdown). Stage 3 can be re-run from the Markdown alone (CLI option --stage 3).
- Placeholders for visuals. Images are removed from the text flow and replaced by tokens such as [IMAGE_PAGE_11_FIG_1], which survive into the JSON as evidence references.
- In-process local models. Models are loaded directly with Hugging Face Transformers on the local GPU. No model server is needed in the default configuration.
- Deterministic fallback. If a model fails to load or returns invalid JSON, the system falls back to the rule-based structurer. The _meta block records which route produced the result.
- Schema-driven output. The target structure is read from vlm_repo/master_schema.json at run time and is placed in the model prompt.

2.5 Coverage of Objective 1 Requirements

The table compares the Objective 1 activities with what the repository implements. "Partial" and "Not present" items are carried into the register in Section 14 and the roadmap in Section 15.

Objective 1 activity	Status	Evidence in the repository
Standard report template and field definitions	Present	Master_Schema_Reference.docx; MASTER_SCHEMA_SPECIFICATION.md
Machine-readable schema (JSON)	Present	vlm_repo/master_schema.json (10 top-level sections)
Secure upload, file validation	Partial	Web upload sanitises the file name; no size limit or type check beyond the browser filter
File hashing, chain-of-custody records	Not present	No hashing or custody log found
Malware scanning workflow	Partial	malware_gate() and classifiers exist; the YARA scanner module and rules are missing from the repository
OCR for scanned documents	Not present	No OCR engine in requirements or code
Text and table extraction	Present	pdfplumber, PyMuPDF, python-docx
Screenshot and evidence extraction	Present	image_offloader
Section detection and field extraction	Present	schema_structurer (rules); unified_vlm_engine (AI)
Rule-based parsing (regex for CVE, CWE, CVSS, IP, URL)	Present	schema_structurer._rule_based_structuring()
AI/ML extraction and synonym mapping	Partial	AI structuring against the schema is present; no explicit synonym dictionary or heading-mapping table found
Linking text and visual evidence	Partial	Image placeholders are carried into proof_of_concept

Objective 1 activity	Status	Evidence in the repository
Source traceability for every field	Partial	Image tokens and the Markdown transcript give traceability; no page, paragraph or table reference per field
Input formats PDF, DOCX, TXT, XLSX, scans, logs	Partial	PDF and DOCX only in the interfaces; a TXT classifier exists but is not wired in
Decoupled report-structure update module	Partial	Schema is an external file read at run time; no versioned update or approval module
Rule derivation and rule-AI documentation	Not present	Rules are in code; no derivation document found
International standards comparison; stakeholder validation report	Not in scope of code	Document deliverables, not software

3. Hardware and Software Requirements

3.1 Hardware

Item	Minimum	Notes
GPU	NVIDIA, 8 GB VRAM	Needed only for the AI engines. Model study figures: about 2.4 GB for Nemotron-Mini and 3.4 GB for Phi-3.5 Vision in 4-bit form
System memory	Not specified in the repository	The rule-based engine is documented at under 150 MB; model loading needs more. Size the host after a trial run
Disk	Several GB per model plus outputs	Model weights are stored under models/. Images and Markdown add to each report's output
CPU	Any modern multi-core	Rule-based engine and layout transcription are CPU-bound
Display	Needed for the GUI only	Web interface and CLI run headless

3.2 Software

Item	Requirement
Operating system	Windows 10/11 or Linux. The GUI launcher is Windows-oriented; the web interface and CLI run on Linux. Development evidence points to both
Python	3.12 was used for this review. The web interface imports the cgi module, which was removed in Python 3.13; use 3.12 or earlier for web_app unless it is updated
CUDA	12.0 or later recommended, with a matching PyTorch build
Tkinter	Needed for the GUI (included with most Windows Python installs; a separate package on many Linux systems)
Ghostscript	Optional; needed only by sanitizer
libmagic	Needed by python-magic for MIME detection in the classifiers

3.3 Deployment Topology

The approved topology is a single processing host inside the air-gapped zone. Models, Python packages and code are prepared and reviewed in a separate staging zone and are carried in through the chain-of-custody process.

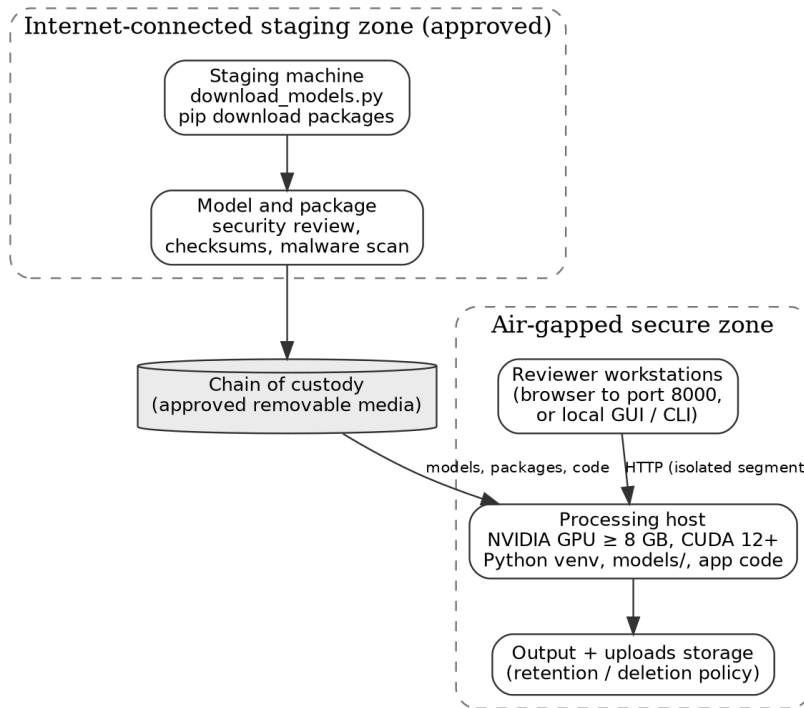


Figure 3: Deployment and offline transfer topology

5. Module Descriptions

5.1 DocumentPipeline

`DocumentPipeline.process_file()` is the orchestrator used by the CLI and the web interface. It derives output paths from the input file name, runs the security gate when enabled, calls the stages in order, and returns a result dictionary with a status of success, error or blocked. Constructor parameters: `backend`, `model`, `schema_path`, `load_in_4bit`, `load_in_8bit`, `enable_security_gate`, `verbose`.

Backend value	Behaviour
<code>vlm_unified</code> (default)	Uses UnifiedVLMEngine for both passes
<code>vlm_direct</code>	Uses VLExtractor to produce JSON directly from page images (GUI mode only; the GUI skips Markdown in this mode)
<code>transformers</code>	Layout transcription, then SchemaStructurer using a Transformers model
<code>rule_based</code>	Layout transcription, then the regex-based structurer. No GPU

5.2 Stage 1: ImageOffloader

- PDF: iterates pages with PyMuPDF, extracts each embedded image, skips images below `min_width` and `min_height` (default 50 pixels), saves PNG files named `page_<n>_fig_<k>.png`, and returns a map of page number to a list of metadata (`fig_idx`, placeholder, filename, path, width, height, format).
- DOCX: reads media files from the zip container and returns a list with placeholders of the form `[IMAGE_DOCX_FIG_<k>]`.
- Vector drawings that are not embedded images are not extracted by this stage.

5.3 Stage 2: Transcription

Layout route

Reads each PDF page with `pdfplumber` and `PyMuPDF` and writes Markdown containing: a page-start comment (`<!-- PAGE_START: n -->`), a heading "Page n", the placeholders for that page's images, each detected table as a Markdown table (cell newlines and pipe characters are escaped), and the page text. DOCX files are read paragraph by paragraph with `python-docx`. The output is saved as `<name>_intermediate.md`.

Vision route (`UnifiedVLMEngine.pass1_visual_transcribe`)

Used with Phi-3.5 Vision. Each page is rendered with a scale factor of 1.5 (about 108 DPI, although a code comment says 150 DPI), passed to the model with the extracted table text, and the model returns Markdown. Generation uses `do_sample=False` and a limit of 1,536 new tokens per page.

5.4 Stage 3: Structuring

AI route (`UnifiedVLMEngine.pass2_structure_json`)

Builds a prompt that contains the full Master Schema and the entire Markdown transcript, runs the model with `do_sample=False` and up to 8,192 new tokens, and parses the first JSON object in the reply. If parsing fails or an exception occurs, the engine calls the rule-based structurer and records `structuring_backend` as `rule_based_fallback`. On success it records `local_vlm_<model>`.

Rule-based route

Parses the Markdown with regular expressions: report metadata, scope tables and URLs, team, timeline, methodology and tools, then one finding per detected observation block. For each finding it extracts the title, severity, status, description, impact, recommendation, CWE and CVE identifiers, CVSS score and vector, affected URLs and image tokens. Summary counts are computed from the findings.

Other routes in SchemaStructurer

- Transformers route: loads a model with `trust_remote_code=True` and generates JSON from the same prompt.
- Ollama route: sends the prompt to an Ollama server at `localhost:11434`. This contradicts the "zero Ollama" statement in the README and needs a separate service; it is reachable only when `backend="ollama"` is passed explicitly, as in `run_vlm_2report`.

5.6 Alternative and Earlier Extractors

V1 extractor (VLExtractor, used for the GUI mode `vlm_direct`) renders page batches, asks Phi-3.5 Vision for JSON directly, merges the results with a deep merge and supports checkpoints. Hybrid extractor and image describer are earlier experiments with a stitched-image approach and a separate image-description step. They are not on the main code path and can be treated as reference code.

5.9 Security Modules

Module	Behaviour
<code>pipeline_integration.malware_gate()</code>	Compiles YARA rules once, scans the file, returns the verdict dictionary, and raises <code>MalwareDetectedError</code> for the verdicts in <code>block_on</code> (default <code>MALICIOUS</code> and <code>SUSPICIOUS</code>). Requires <code>yara_scanner</code> (<code>compile_rules</code> , <code>scan_file</code> , <code>DEFAULT_RULES_DIR</code>), which is not in the repository
<code>docx_classifier</code>	Opens the DOCX as a zip; flags <code>vbaProject.bin</code> (macro), external relationship targets (template injection) and embedded OLE objects; returns <code>PASS</code> , <code>SANITIZE</code> or <code>BLOCK</code>
<code>embed_classifier</code> / <code>txtclassifier</code>	Identify MIME type with <code>python-magic</code> and compare with an allow-list or deny-list
<code>sanitizer.ghostscript_sanitize()</code>	Re-writes a PDF through Ghostscript to remove JavaScript, actions and embedded executables; returns <code>None</code> if gs is absent
<code>docx_sanitizer.docx_sanitize()</code>	Re-writes the DOCX zip without macros, OLE objects and external links
<code>deep_analyzer</code>	Second-stage content analysis for files in a REVIEW bucket; imports <code>pdf_content_classifier</code> , <code>js_classifier</code> , <code>pdf_page_locator</code> and <code>yara_scanner</code> , none of which are in the repository

6. Data Design

6.1 Master Schema

vlm_repo/master_schema.json defines the output as ten top-level sections. A field-level table is in Appendix B; the rationale for each field is in MASTER_SCHEMA_SPECIFICATION.md, which also links fields to the benchmarking criteria used in later objectives. The schema file is a template of example values and type hints (for example "High | Medium | Low", "YYYY-MM-DD"), not a JSON Schema validation document, so the jsonschema library listed in requirements.txt is not used to enforce it.

6.2 Metadata Block

Each output carries a `_meta` object added by the pipeline. It is the only record of how the file was produced and must be preserved.

Key	Meaning
pipeline_version	Code version that wrote the file (values seen: 3.0.0-decoupled, 3.1.0-decoupled, 3.2.0-local-vm)
source_file	Original report file name
structuring_backend	local_vlm_<model>, rule_based or rule_based_fallback
structuring_model / model	Model identifier or rule_based_engine

7. Dependencies and Third-Party Components

7.1 Python Packages

Package	Declared	Use
python-docx	>=1.1.0	Read DOCX text and tables
pdfplumber	>=0.10.0	PDF text and table extraction
pypdf	>=4.0.0	PDF analysis (used by the deep analyser)
pymupdf	>=1.24.0	PDF rendering and image extraction (imported as fitz)
jsonschema	>=4.21.0	Declared; imported optionally by schema structurer, not used to validate output
requests	>=2.31.0	HTTP client; used only by the Ollama route
click	>=8.1.0	CLI
python-dotenv	>=1.0.0	Load .env
torch, torchvision	>=2.1.0, >=0.16.0	Model execution on GPU
transformers	>=4.45.0	Model and tokenizer loading
accelerate	>=0.25.0	Device placement
bitsandbytes	>=0.41.0	4-bit NF4 quantisation
Pillow	>=10.0.0	Image handling
tqdm	>=4.66.0	Progress bars
yara-python	>=3.11.0	YARA rule engine for the security gate
python-magic	>=0.4.27	MIME detection (needs libmagic)

7.2 Models

Model		Hugging Face identifier	Role
Nemotron-Mini Instruct	4B	nvidia/Nemotron-Mini-4B-Instruct	Text model for Stage 3 structuring
Phi-3.5 Vision Instruct		microsoft/Phi-3.5-vision-instruct	Page-image transcription (pass 1) and structuring (pass 2)
Nemotron (registry only)	70B	nvidia/Llama-3.1-Nemotron-70B-Instruct-HF	Listed in the registry; too large for the specified GPU

8. Installation and Configuration

8.1 Build on the Machine

1. Create a virtual environment on a machine with the same operating system and Python version as the target host.
2. Download wheels for all packages: `pip download -r requirements.txt -d wheelhouse`. Select the PyTorch build that matches the target CUDA version.
3. Download the models: `python download_models --model nemotron-mini --local-dir <dir>`, and the same for phi-3.5 if required.
4. Record SHA-256 checksums of every file; run the model security review (source code, custom code, weights, dependencies) and a malware scan.
5. Package code, wheelhouse and models; transfer by the authorised chain of custody.

9. Interfaces

9.1 Command-Line Interface

Entry point: `python -m vlm_repo.src.pipeline`. Options: `--input`, `--input-dir`, `--output`, `--output-dir`, `--stage` (1, 2, 3, all), `--backend` (vlm_unified, vlm_direct, transformers, rule_based), `--model`, `--image-storage-dir`, `--dry-run`, `--keep-markdown/--no-keep-markdown`. Exit behaviour: a result status of error or blocked is printed to the console. Batch mode processes every supported file in `--input-dir`.

9.2 Web Interface

Method	Path	Description
GET	/ or /index.html	Returns the embedded HTML page
GET	/api/health	Returns a health response
POST	/api/process	multipart/form-data with fields file (required) and model (optional; default nvidia/Nemotron-Mini-4B-Instruct; the value rule_based selects the rule-based backend)

Responses of POST /api/process

HTTP	Body
200	Pipeline result dictionary (status success, with paths, counts and timings), or {"status":"blocked","error":"File blocked by Security Pre-Gate: ..."} }
400	{"status":"error","error":"Expected multipart/form-data"} or "No file uploaded"
404	Unknown path
500	{"status":"error","error":"<exception text>"}

There is no authentication, no CSRF protection, no request size limit and no concurrency control. The cgi module used to parse uploads is deprecated.

10. Security Architecture

10.1 Controls Present

Control	Implementation
Local-only inference	In-process Transformers; no cloud API in the default path
Pre-parse malware gate	malware_gate() before any parser runs (when its module is present)
Active-content inspection	DOCX macro, external-reference and OLE detection; embedded-file MIME checks
Sanitisation	Ghostscript PDF rewrite; DOCX zip rewrite
Upload name sanitising	Web handler replaces characters outside letters, digits, underscore, hyphen and dot
Deterministic decoding	do_sample=False for model generation

11. Performance and Capacity

Engine	Reported speed	Notes
Rule-based	Under 2 seconds per document	CPU only; model study figure
Nemotron-Mini (Stage 3)	About 5 to 12 seconds	8 GB RTX-class GPU; excludes model loading
Phi-3.5 Vision (Stage 2)	About 15 to 30 seconds	Figure from the model study; scales with page count

- The model is loaded again for every file in the web interface and for every GUI run, which adds start-up time that the table does not include.
- One page is processed at a time in pass 1; MAX_WORKERS in .env is defined but was not found to be read by the code.
- No limit is placed on page count or file size. Very large reports increase memory use during transcription and may exceed the model context in Stage 3.
- The sample set in VAPT-API/ (six reports) and the stored outputs can be used as a baseline for timing on the target hardware.

12. Testing and Verification

12.1 Automated Tests

File	Coverage
tests/test_objective1	Pytest suite for ImageOffloader, LosslessTranscriber, the DOCX / TXT / embedded-file classifiers and the DOCX / PDF sanitisers
tests/test_local_vlm	Local VLM engine checks (requires models)
test_run_pipeline	Simple pipeline invocation script

12.2 Sample Data

VAPT-API/ holds six sample PDFs; outputs/ and result_json/ hold their generated JSON for the folders VAPT-API, Mobile application, Network Report, iot and best. These are examples from earlier runs, produced by different pipeline versions, and should be regenerated before being used as expected results.

Appendix A: Environment Variables

Variable	Default / example	Effect
PIPELINE_BACKEND	vlm_unified	Default backend for CLI and web
VLM_MODEL	nvidia/Nemotron-Mini-4B-Instruct	Default model identifier
LOAD_IN_4BIT	True	Defined in .env.example; the GUI uses its own checkbox, the pipeline uses its constructor argument
PIPELINE_LOG_LEVEL	INFO	Declared; not read by the code reviewed
MAX_WORKERS	4	Declared; not read by the code reviewed
LLM_MODEL	not set	Alternative model name read by schema structurer
VLM_MODELS_DIR	not set	Extra folder searched for model files
HF_HUB_OFFLINE, TRANSFORMERS_OFFLINE	recommended: 1	Hugging Face switches that block downloads (to be set by the administrator)

Appendix B: Master Schema Field Reference

Section	Main fields
report_meta	report_id, report_title, report_type, report_release_date, report_version, audit_type, audit_period (from, to), classification
document_control	document_title, document_id, document_version, prepared_by, reviewed_by, approved_by, released_by, release_date, change_history[], distribution_list[]
engagement_scope	assets[] (sr_no, asset_description, url, criticality, internal_ip, public_ip, in_scope_functions), scope_updated_till, exclusions[], testing_credentials[] (role, account_identifier, auth_type, purpose)
auditing_team	List of team members
audit_timeline	assessment_start_date, assessment_end_date
methodology	standards_referred[]
tools_used	List of tools
executive_summary	total_findings, severity_count, narrative, summary_table[]
detailed_observations[]	finding_id, title, status, severity, impact_level, likelihood_level, cve_id[], cvss_score, cvss_vector, severity_rationale, reported_epss_score, observation_date, cwe[], affected_asset[], regulatory_references[], observation_description, preconditions, proof_of_concept[] (sr_no, type, reference, capture_date, is_retest, redacted, description), impact, likely_root_cause, recommendation, reference, new_or_repeat
appendices	risk_rating_criteria[], retest_closure[]

Appendix C: Glossary

Term	Meaning
Air-gapped	Environment with no connection to external networks
CUDA	NVIDIA platform for GPU computing
Context window	Maximum amount of text a model can read in one prompt
Intermediate Markdown	Stage 2 transcript between the report and the JSON
Master Schema	Standard JSON structure for an audit report
NF4	4-bit NormalFloat quantisation used by bitsandbytes
Pre-gate	Security scan performed before any parser opens the file
Provenance	Record of where in the source a value came from
Quantisation	Storing model weights with fewer bits to save memory
VLM	Vision-language model
YARA	Rule language for identifying malicious file patterns